Chapter 9. Process Address Space

As seen in the previous chapter, a kernel function gets dynamic memory in a fairly straightforward manner by invoking one of a variety of functions: `_ _get_free_pages( )` or `alloc_pages( )` to get pages from the zoned page frame allocator, `kmem_cache_alloc( )` or `kmalloc( )` to use the slab allocator for specialized or general-purpose objects, and `vmalloc( )` or `vmalloc_32( )` to get a noncontiguous memory area. If the request can be satisfied, each of these functions returns a page descriptor address or a linear address identifying the beginning of the allocated dynamic memory area.

These simple approaches work for two reasons:

- The kernel is the highest-priority component of the operating system. If a kernel function makes a request for dynamic memory, it must have a valid reason to issue that request, and there is no point in trying to defer it.
- The kernel trusts itself. All kernel functions are assumed to be error-free, so the kernel does not need to insert any protection against programming errors.

When allocating memory to User Mode processes, the situation is entirely different:

- Process requests for dynamic memory are considered non-urgent. When a process's executable file is loaded, for instance, it is unlikely that the process will address all the pages of code in the near future. Similarly, when a process invokes `malloc( )` to get additional dynamic memory, it doesn't mean the process will soon access all the additional memory obtained. Thus, as a general rule, the kernel tries to defer allocating dynamic memory to User Mode processes.
- Because user programs cannot be trusted, the kernel must be prepared to catch all addressing errors caused by processes in User Mode.

As this chapter describes, the kernel succeeds in deferring the allocation of dynamic memory to processes by using a new kind of resource. When a User Mode process asks for dynamic memory, it doesn't get additional page frames; instead, it gets the right to use a new range of linear addresses, which become part of its address space. This interval is called a "memory region."

In the next section, we discuss how the process views dynamic memory. We then describe the basic components of the process address space in the section "Memory Regions." Next, we examine in detail the role played by the Page Fault exception handler in deferring the allocation of page frames to processes and illustrate how the kernel creates and deletes whole process address spaces. Last, we discuss the APIs and system calls related to address space management.

9.1. The Process's Address Space

The address space of a process consists of all linear addresses that the process is allowed to use. Each process sees a different set of linear addresses; the address used by one process bears no relation to the address used by another. As we will see later, the kernel may dynamically modify a process address space by adding or removing intervals of linear addresses.

The kernel represents intervals of linear addresses by means of resources called memory regions , which are characterized by an initial linear address, a length, and some access rights. For reasons of efficiency, both the initial address and the length of a memory region must be multiples of 4,096, so that the data identified by each memory region completely fills up the page frames allocated to it. Following are some typical situations in which a process gets new memory regions:

- When the user types a command at the console, the shell process creates a new process to execute the command. As a result, a fresh address space, and thus a set of memory regions, is assigned to the new process (see the section "Creating and Deleting a Process Address Space" later in this chapter; also, see Chapter 20).
- A running process may decide to load an entirely different program. In this case, the process ID remains unchanged, but the memory regions used before loading the program are released and a new set of memory regions is assigned to the process (see the section "The exec Functions" in Chapter 20).
- A running process may perform a "memory mapping" on a file (or on a portion of it). In such cases, the kernel assigns a new memory region to the process to map the file (see the section "Memory Mapping" in Chapter 16).
- A process may keep adding data on its User Mode stack until all addresses in the memory region that map the stack have been used. In this case, the kernel may decide to expand the size of that memory region (see the section "Page Fault Exception Handler" later in this chapter).
- A process may create an IPC-shared memory region to share data with other cooperating processes. In this case, the kernel assigns a new memory region to the process to implement this construct (see the section "IPC Shared Memory" in Chapter 19).
- A process may expand its dynamic area (the heap) through a function such as `malloc( )`. As a result, the kernel may decide to expand the size of the memory region assigned to the heap (see the section "Managing the Heap" later in this chapter).

Table 9-1 illustrates some of the system calls related to the previously mentioned tasks. `brk( )` is discussed at the end of this chapter, while the remaining system calls are described in other chapters.

Table 9-1. System calls related to memory region creation and deletion

System call	Description
`brk( )`	Changes the heap size of the process
`execve( )`	Loads a new executable file, thus changing the process

	address space
`_exit( )`	Terminates the current process and destroys its address space
`fork( )`	Creates a new process, and thus a new address space
`mmap( ),mmap2( )`	Creates a memory mapping for a file, thus enlarging the process address space
`mremap( )`	Expands or shrinks a memory region
`remap_file_pages( )`	Creates a non-linear mapping for a file (see Chapter 16)
`munmap( )`	Destroys a memory mapping for a file, thus contracting the process address space
`shmat( )`	Attaches a shared memory region
`shmdt( )`	Detaches a shared memory region

As we'll see in the later section "Page Fault Exception Handler," it is essential for the kernel to identify the memory regions currently owned by a process (the address space of a process), because that allows the Page Fault exception handler to efficiently distinguish between two types of invalid linear addresses that cause it to be invoked:

- Those caused by programming errors.
- Those caused by a missing page; even though the linear address belongs to the process's address space, the page frame corresponding to that address has yet to be allocated.

The latter addresses are not invalid from the process's point of view; the induced Page Faults are exploited by the kernel to implement demand paging : the kernel provides the missing page frame and lets the process continue.

9.2. The Memory Descriptor

All information related to the process address space is included in an object called the memory descriptor of type `mm_struct`. This object is referenced by the `mm` field of the process descriptor. The fields of a memory descriptor are listed in Table 9-2.

Table 9-2. The fields of the memory descriptor

Type	Field	Description
`struct vm_area_struct *`	`mmap`	Pointer to the head of the list of memory region objects
`struct rb_root`	`mm_rb`	Pointer to the root of the red-black tree of memory region objects
`struct vm_area_struct *`	`mmap_cache`	Pointer to the last referenced memory region object
unsigned long (*)( )	get_unmapped_area	Method that searches an available linear address interval in the process address space
void (*)( )	unmap_area	Method invoked when releasing a linear address interval
unsigned long	mmap_base	Identifies the linear address of the first allocated anonymous memory region or file memory mapping (see the section "Program Segments and Process Memory Regions" in Chapter 20)
unsigned long	free_area_cache	Address from which the kernel will look for a free interval of linear addresses in the process address space
`pgd_t *`	`pgd`	Pointer to the Page Global Directory
`atomic_t`	`mm_users`	Secondary usage counter
`atomic_t`	`mm_count`	Main usage counter
`int`	`map_count`	Number of memory regions
`struct rw_semaphore`	`mmap_sem`	Memory regions' read/write semaphore
`spinlock_t`	`page_table_lock`	Memory regions' and Page Tables' spin lock
`struct list_head`	`mmlist`	Pointers to adjacent elements in the list of memory descriptors
`unsigned long`	`start_code`	Initial address of executable code
`unsigned long`	`end_code`	Final address of executable code
`unsigned long`	`start_data`	Initial address of initialized data
`unsigned long`	`end_data`	Final address of initialized data
`unsigned long`	`start_brk`	Initial address of the heap
`unsigned long`	`brk`	Current final address of the heap

unsigned long	start_stack	Initial address of User Mode stack
unsigned long	arg_start	Initial address of command-line arguments
unsigned long	arg_end	Final address of command-line arguments
unsigned long	env_start	Initial address of environment variables
unsigned long	env_end	Final address of environment variables
unsigned long	rss	Number of page frames allocated to the process
unsigned long	anon_rss	Number of page frames assigned to anonymous memory mappings
unsigned long	total_vm	Size of the process address space (number of pages)
unsigned long	locked_vm	Number of "locked" pages that cannot be swapped out (see Chapter 17)
unsigned long	shared_vm	Number of pages in shared file memory mappings
unsigned long	exec_vm	Number of pages in executable memory mappings
unsigned long	stack_vm	Number of pages in the User Mode stack
unsigned long	reserved_vm	Number of pages in reserved or special memory regions
unsigned long	def_flags	Default access flags of the memory regions
unsigned long	nr_ptes	Number of Page Tables of this process
unsigned long []	saved_auxv	Used when starting the execution of an ELF program (see Chapter 20)
unsigned int	dumpable	Flag that specifies whether the process can produce a core dump of the memory
cpumask_t	cpu_vm_mask	Bit mask for lazy TLB switches (see Chapter 2)
mm_context_t	context	Pointer to table for architecture-specific information (e.g., LDT's address in 80 86 platforms)
unsigned long	swap_token_time	When this process will become eligible for having the swap token (see the section "The Swap Token" in Chapter 17)
char	recent_pagein	Flag set if a major Page Fault has recently occurred
int	core_waiters	Number of lightweight processes that are dumping the contents of the process address space to a core file (see the section "Deleting a Process Address Space" later in this chapter)
struct completion *	core_startup_done	Pointer to a completion used when creating a core file (see the section "Completions" in Chapter 5)
struct completion	core_done	Completion used when creating a core file
rwlock_t	ioctx_list_lock	Lock used to protect the list of asynchronous I/O contexts (see Chapter

		16)
struct kioctx *	ioctx_list	List of asynchronous I/O contexts (see Chapter 16)
struct kioctx	default_kioctx	Default asynchronous I/O context (see Chapter 16)
unsigned long	hiwater_rss	Maximum number of page frames ever owned by the process
unsigned long	hiwater_vm	Maximum number of pages ever included in the memory regions of the process

All memory descriptors are stored in a doubly linked list. Each descriptor stores the address of the adjacent list items in the `mmlist` field. The first element of the list is the `mmlist` field of `init_mm`, the memory descriptor used by process 0 in the initialization phase. The list is protected against concurrent accesses in multiprocessor systems by the `mmlist_lock` spin lock.

The `mm_users` field stores the number of lightweight processes that share the `mm_struct` data structure (see the section "The clone( ), fork( ), and vfork( ) System Calls" in Chapter 3). The `mm_count` field is the main usage counter of the memory descriptor; all "users" in `mm_users` count as one unit in `mm_count`. Every time the `mm_count` field is decreased, the kernel checks whether it becomes zero; if so, the memory descriptor is deallocated because it is no longer in use.

We'll try to explain the difference between the use of `mm_users` and `mm_count` with an example. Consider a memory descriptor shared by two lightweight processes. Normally, its `mm_users` field stores the value 2, while its `mm_count` field stores the value 1 (both owner processes count as one).

If the memory descriptor is temporarily lent to a kernel thread (see the next section), the kernel increases the `mm_count` field. In this way, even if both lightweight processes die and the `mm_users` field becomes zero, the memory descriptor is not released until the kernel thread finishes using it because the `mm_count` field remains greater than zero.

If the kernel wants to be sure that the memory descriptor is not released in the middle of a lengthy operation, it might increase the `mm_users` field instead of `mm_count` (this is what the `try_to_unuse( )` function does; see the section "Activating and Deactivating a Swap Area" in Chapter 17). The final result is the same because the increment of `mm_users` ensures that `mm_count` does not become zero even if all lightweight processes that own the memory descriptor die.

The `mm_alloc( )` function is invoked to get a new memory descriptor. Because these descriptors are stored in a slab allocator cache, `mm_alloc( )` calls `kmem_cache_alloc( )`, initializes the new memory descriptor, and sets the `mm_count` and `mm_users` field to 1.

Conversely, the `mmput( )` function decreases the `mm_users` field of a memory descriptor. If that field becomes 0, the function releases the Local Descriptor Table, the memory region descriptors (see later in this chapter), and the Page Tables referenced by the memory descriptor, and then invokes `mmdrop( )`. The latter function decreases `mm_count` and, if it becomes zero, releases the `mm_struct` data structure.

The `mmap`, `mm_rb`, `mmlist`, and `mmap_cache` fields are discussed in the next section.

9.2.1. Memory Descriptor of Kernel Threads

Kernel threads run only in Kernel Mode, so they never access linear addresses below TASK_SIZE (same as PAGE_OFFSET, usually 0xc0000000). Contrary to regular processes, kernel threads do not use memory regions, therefore most of the fields of a memory descriptor are meaningless for them.

Because the Page Table entries that refer to the linear address above TASK_SIZE should always be identical, it does not really matter what set of Page Tables a kernel thread uses. To avoid useless TLB and cache flushes, a kernel thread uses the set of Page Tables of the last previously running regular process. To that end, two kinds of memory descriptor pointers are included in every process descriptor: mm and active_mm.

The mm field in the process descriptor points to the memory descriptor owned by the process, while the active_mm field points to the memory descriptor used by the process when it is in execution. For regular processes, the two fields store the same pointer. Kernel threads, however, do not own any memory descriptor, thus their mm field is always NULL. When a kernel thread is selected for execution, its active_mm field is initialized to the value of the active_mm of the previously running process (see the section "The schedule( ) Function" in Chapter 7).

There is, however, a small complication. Whenever a process in Kernel Mode modifies a Page Table entry for a "high" linear address (above TASK_SIZE), it should also update the corresponding entry in the sets of Page Tables of all processes in the system. In fact, once set by a process in Kernel Mode, the mapping should be effective for all other processes in Kernel Mode as well. Touching the sets of Page Tables of all processes is a costly operation; therefore, Linux adopts a deferred approach.

We already mentioned this deferred approach in the section "Noncontiguous Memory Area Management" in Chapter 8: every time a high linear address has to be remapped (typically by vmalloc( ) or vfree( )), the kernel updates a canonical set of Page Tables rooted at the swapper_pg_dir master kernel Page Global Directory (see the section "Kernel Page Tables" in Chapter 2). This Page Global Directory is pointed to by the pgd field of a master memory descriptor , which is stored in the init_mm variable.[*]

> [*] We mentioned in the section "Kernel Threads" in Chapter 3 that the swapper process uses init_mm during the initialization phase. However, swapper never uses this memory descriptor once the initialization phase completes.

Later, in the section "Handling Noncontiguous Memory Area Accesses," we'll describe how the Page Fault handler takes care of spreading the information stored in the canonical Page Tables when effectively needed.

9.3. Memory Regions

Linux implements a memory region by means of an object of type `vm_area_struct`; its fields are shown in Table 9-3.[*]

[*] We omitted describing a few additional fields used in NUMA systems.

Table 9-3. The fields of the memory region object

Type	Field	Description
`struct mm_struct *`	`vm_mm`	Pointer to the memory descriptor that owns the region.
`unsigned long`	`vm_start`	First linear address inside the region.
`unsigned long`	`vm_end`	First linear address after the region.
`struct vm_area_struct *`	`vm_next`	Next region in the process list.
`pgprot_t`	`vm_page_prot`	Access permissions for the page frames of the region.
`unsigned long`	`vm_flags`	Flags of the region.
`struct rb_node`	`vm_rb`	Data for the red-black tree (see later in this chapter).
`union`	`shared`	Links to the data structures used for reverse mapping (see the section "Reverse Mapping for Mapped Pages" in Chapter 17).
struct list_head	anon_vma_node	Pointers for the list of anonymous memory regions (see the section "Reverse Mapping for Anonymous Pages" in Chapter 17).
struct anon_vma *	anon_vma	Pointer to the `anon_vma` data structure (see the section "Reverse Mapping for Anonymous Pages" in Chapter 17).
`struct vm_operations_struct*`	`vm_ops`	Pointer to the methods of the memory region.
`unsigned long`	`vm_pgoff`	Offset in mapped file (see Chapter 16). For anonymous pages, it is either zero or equal to `vm_start/PAGE_SIZE` (see Chapter 17).
`struct file *`	`vm_file`	Pointer to the file object of the mapped file, if any.
`void *`	`vm_private_data`	Pointer to private data of the memory region.
unsigned long	vm_truncate_count	Used when releasing a linear address interval in a non-linear file memory

mapping.

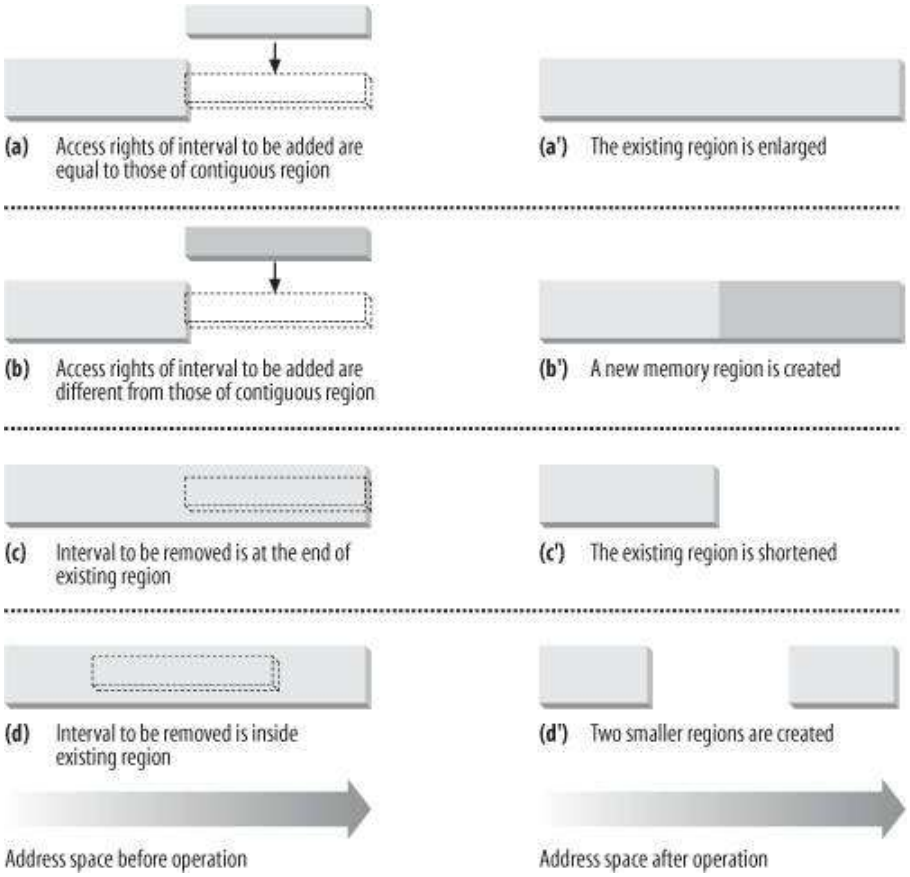
Each memory region descriptor identifies a linear address interval. The `vm_start` field contains the first linear address of the interval, while the `vm_end` field contains the first linear address outside of the interval; `vm_end-vm_start` thus denotes the length of the memory region. The `vm_mm` field points to the `mm_struct` memory descriptor of the process that owns the region. We will describe the other fields of `vm_area_struct` as they come up.

Memory regions owned by a process never overlap, and the kernel tries to merge regions when a new one is allocated right next to an existing one. Two adjacent regions can be merged if their access rights match.

As shown in Figure 9-1, when a new range of linear addresses is added to the process address space, the kernel checks whether an already existing memory region can be enlarged (case a). If not, a new memory region is created (case b). Similarly, if a range of linear addresses is removed from the process address space, the kernel resizes the affected memory regions (case c). In some cases, the resizing forces a memory region to split into two smaller ones (case d) .[*]

[*] Removing a linear address interval may theoretically fail because no free memory is available for a new memory descriptor.

Figure 9-1. Adding or removing a linear address interval



The `vm_ops` field points to a `vm_operations_struct` data structure, which stores the methods of the

memory region. Only four methodsillustrated in Table 9-4are applicable to UMA systems.

Table 9-4. The methods to act on a memory region

Method	Description
`open`	Invoked when the memory region is added to the set of regions owned by a process.
`close`	Invoked when the memory region is removed from the set of regions owned by a process.
`nopage`	Invoked by the Page Fault exception handler when a process tries to access a page not present in RAM whose linear address belongs to the memory region (see the later section "Page Fault Exception Handler").
`populate`	Invoked to set the page table entries corresponding to the linear addresses of the memory region (prefaulting). Mainly used for non-linear file memory mappings.

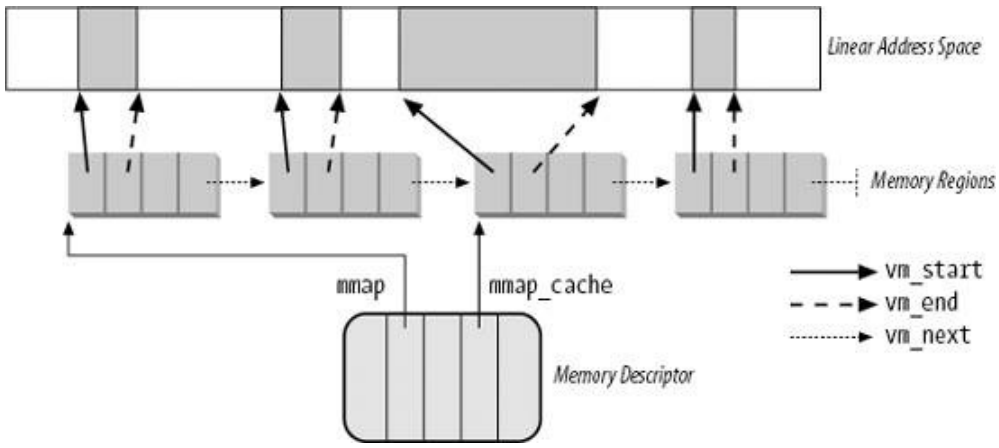
9.3.1. Memory Region Data Structures

All the regions owned by a process are linked in a simple list. Regions appear in the list in ascending order by memory address; however, successive regions can be separated by an area of unused memory addresses. The `vm_next` field of each `vm_area_struct` element points to the next element in the list. The kernel finds the memory regions through the `mmap` field of the process memory descriptor, which points to the first memory region descriptor in the list.

The `map_count` field of the memory descriptor contains the number of regions owned by the process. By default, a process may own up to 65,536 different memory regions; however, the system administrator may change this limit by writing in the */proc/sys/vm/max_map_count* file.

Figure 9-2 illustrates the relationships among the address space of a process, its memory descriptor, and the list of memory regions.

Figure 9-2. Descriptors related to the address space of a process



A frequent operation performed by the kernel is to search the memory region that includes a specific linear address. Because the list is sorted, the search can terminate as soon as a memory region that ends after the specific linear address is found.

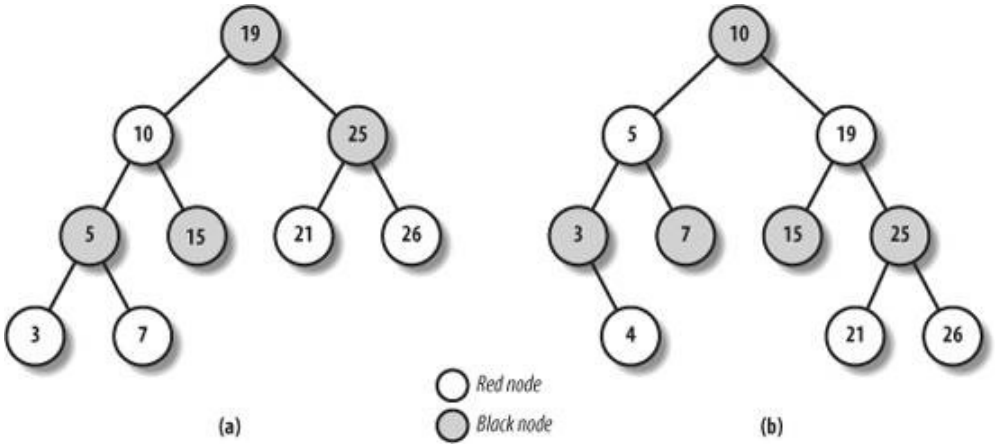
However, using the list is convenient only if the process has very few memory regionslet's say less than a few tens of them. Searching, inserting elements, and deleting elements in the list involve a number of operations whose times are linearly proportional to the list length.

Although most Linux processes use very few memory regions, there are some large applications, such as object-oriented databases or specialized debuggers for the usage of `malloc()`, that have many hundreds or even thousands of regions. In such cases, the memory region list management becomes very inefficient, hence the performance of the memory-related system calls degrades to an intolerable point.

Therefore, Linux 2.6 stores memory descriptors in data structures called red-black trees . In a red-black tree, each element (or node) usually has two children: a left child and a right child. The elements in the tree are sorted. For each node N, all elements of the subtree rooted at the left child of N precede N, while, conversely, all elements of the subtree rooted at the right child of N follow N (see Figure 9-3(*a*); the key of the node is written inside the node itself. Moreover, a red-black tree must satisfy four additional rules:

1. Every node must be either red or black.
2. The root of the tree must be black.
3. The children of a red node must be black.
4. Every path from a node to a descendant leaf must contain the same number of black nodes . When counting the number of black nodes, null pointers are counted as black nodes.

Figure 9-3. Example of red-black trees



These four rules ensure that every red-black tree with n internal nodes has a height of at most 2 x log(n + 1).

Searching an element in a red-black tree is thus very efficient, because it requires operations whose execution time is linearly proportional to the logarithm of the tree size. In other words, doubling the number of memory regions adds just one more iteration to the operation.

Inserting and deleting an element in a red-black tree is also efficient, because the algorithm can quickly traverse the tree to locate the position at which the element will be inserted or from which it will be removed. Each new node must be inserted as a leaf and colored red. If the operation breaks the rules, a few nodes of the tree must be moved or recolored.

For instance, suppose that an element having the value 4 must be inserted in the red-black tree shown in Figure 9-3(a). Its proper position is the right child of the node that has key 3, but once it is inserted, the red node that has the value 3 has a red child, thus breaking rule 3. To satisfy the rule, the color of nodes that have the values 3, 4, and 7 is changed. This operation, however, breaks rule 4, thus the algorithm performs a "rotation" on the subtree rooted at the node that has the key 19, producing the new red-black tree shown in Figure 9-3(b). This looks complicated, but inserting or deleting an element in a red-black tree requires a small number of operationsa number linearly proportional to the logarithm of the tree size.

Therefore, to store the memory regions of a process, Linux uses both a linked list and a red-black tree. Both data structures contain pointers to the same memory region descriptors. When inserting or removing a memory region descriptor, the kernel searches the previous and next elements through the red-black tree and uses them to quickly update the list without scanning it.

The head of the linked list is referenced by the `mmap` field of the memory descriptor. Each memory region object stores the pointer to the next element of the list in the `vm_next` field. The head of the red-black tree is referenced by the `mm_rb` field of the memory descriptor. Each memory region object stores the color of the node, as well as the pointers to the parent, the left child, and the right child, in the `vm_rb` field of type `rb_node`.

In general, the red-black tree is used to locate a region including a specific address, while the linked list is mostly useful when scanning the whole set of regions.

9.3.2. Memory Region Access Rights

Before moving on, we should clarify the relation between a page and a memory region. As mentioned in Chapter 2, we use the term "page" to refer both to a set of linear addresses and to the data contained in this group of addresses. In particular, we denote the linear address interval ranging between 0 and 4,095 as page 0, the linear address interval ranging between 4,096 and 8,191 as page 1, and so forth. Each memory region therefore consists of a set of pages that have consecutive page numbers.

We have already discussed two kinds of flags associated with a page:

- A few flags such as `Read/Write`, `Present`, or `User/Supervisor` stored in each Page Table entry (see the section "Regular Paging" in Chapter 2).
- A set of flags stored in the `flags` field of each `page` descriptor (see the section "Page Frame Management" in Chapter 8).

The first kind of flag is used by the 80 x 86 hardware to check whether the requested kind of addressing can be performed; the second kind is used by Linux for many different purposes (see Table 8-2).

We now introduce a third kind of flag: those associated with the pages of a memory region. They are stored in the `vm_flags` field of the `vm_area_struct` descriptor (see Table 9-5). Some flags offer the kernel information about all the pages of the memory region, such as what they contain and what rights the process has to access each page. Other flags describe the region itself, such as how it can grow.

Table 9-5. The memory region flags

Flag name

Description

`VM_READ`

Pages can be read

`VM_WRITE`

Pages can be written

`VM_EXEC`

Pages can be executed

`VM_SHARED`

Pages can be shared by several processes

`VM_MAYREAD`

`VM_READ` flag may be set

`VM_MAYWRITE`

`VM_WRITE` flag may be set

`VM_MAYEXEC`

`VM_EXEC` flag may be set

`VM_MAYSHARE`

`VM_SHARE` flag may be set

`VM_GROWSDOWN`

The region can expand toward lower addresses

`VM_GROWSUP`

The region can expand toward higher addresses

`VM_SHM`

The region is used for IPC's shared memory

`VM_DENYWRITE`

The region maps a file that cannot be opened for writing

`VM_EXECUTABLE`

The region maps an executable file

`VM_LOCKED`

Pages in the region are locked and cannot be swapped out

`VM_IO`

The region maps the I/O address space of a device

`VM_SEQ_READ`

The application accesses the pages sequentially

`VM_RAND_READ`

The application accesses the pages in a truly random order

`VM_DONTCOPY`

Do not copy the region when forking a new process

`VM_DONTEXPAND`

Forbid region expansion through `mremap( )` system call

`VM_RESERVED`

The region is special (for instance, it maps the I/O address space of a device), so its pages must not be swapped out

`VM_ACCOUNT`

Check whether there is enough free memory for the mapping when creating an IPC shared memory region (see Chapter 19)

`VM_HUGETLB`

The pages in the region are handled through the extended paging mechanism (see the section "Extended Paging" in Chapter 2)

`VM_NONLINEAR`

The region implements a non-linear file mapping

Page access rights included in a memory region descriptor may be combined arbitrarily. It is possible, for instance, to allow the pages of a region to be read but not executed. To implement this protection scheme efficiently, the Read, Write, and Execute access rights associated with the pages of a memory region must be duplicated in all the corresponding Page Table entries, so that checks can be directly performed by the Paging Unit circuitry. In other words, the page access rights dictate what kinds of access should generate a Page Fault exception. As we'll see shortly, the job of figuring out what caused the Page Fault is delegated by Linux to the Page Fault handler, which implements several page-handling strategies.

The initial values of the Page Table flags (which must be the same for all pages in the memory region, as we have seen) are stored in the `vm_ page_ prot` field of the `vm_area_struct` descriptor. When adding a page, the kernel sets the flags in the corresponding Page Table entry according to the value of the `vm_ page_ prot` field.

However, translating the memory region's access rights into the page protection bits is not straightforward for the following reasons:

- In some cases, a page access should generate a Page Fault exception even when its access type is granted by the page access rights specified in the `vm_flags` field of the corresponding memory region. For instance, as we'll see in the section "Copy On Write" later in this chapter, the kernel may wish to store two identical, writable private pages (whose `VM_SHARE` flags are cleared) belonging to two different processes in the same page frame; in this case, an exception should be generated when either one of the processes tries to modify the page.
- As mentioned in Chapter 2, 80 x 86 processors's Page Tables have just two protection bits, namely the `Read/Write` and `User/Supervisor` flags. Moreover, the `User/Supervisor` flag of every page included in a memory region must always be set, because the page must always be accessible by User Mode processes.
- Recent Intel Pentium 4 microprocessors with PAE enabled sport a `NX` (No eXecute) flag in each 64-bit Page Table entry.

If the kernel has been compiled without support for PAE, Linux adopts the following rules, which overcome the hardware limitation of the 80 x 86 microprocessors:

- The Read access right always implies the Execute access right, and vice versa.
- The Write access right always implies the Read access right.

Conversely, if the kernel has been compiled with support for PAE and the CPU has the `NX` flag, Linux adopts different rules:

- The Execute access right always implies the Read access right.
- The Write access right always implies the Read access right.

Moreover, to correctly defer the allocation of page frames through the "Copy On Write" technique (see later in this chapter), the page frame is write-protected whenever the corresponding page must not be shared by several processes.

Therefore, the 16 possible combinations of the Read, Write, Execute, and Share access rights are scaled down according to the following rules:

- If the page has both Write and Share access rights, the `Read/Write` bit is set.
- If the page has the Read or Execute access right but does not have either the Write or the Share access right, the `Read/Write` bit is cleared.
- If the `NX` bit is supported and the page does not have the Execute access right, the `NX` bit is set.
- If the page does not have any access rights, the `Present` bit is cleared so that each access generates a Page Fault exception. However, to distinguish this condition from the real page-not-present case, Linux also sets the `Page size` bit to 1.[*]

> [*] You might consider this use of the `Page size` bit to be a dirty trick, because the bit was meant to indicate the real page size. But Linux can get away with the deception because the 80 x 86 chip checks the `Page size` bit in Page Directory entries, but not in Page Table entries.

The downscaled protection bits corresponding to each combination of access rights are stored in the 16 elements of the `protection_map` array.

9.3.3. Memory Region Handling

Having the basic understanding of data structures and state information that control memory handling , we can look at a group of low-level functions that operate on memory region descriptors. They should be considered auxiliary functions that simplify the implementation of `do_mmap( )` and `do_munmap( )`. Those two functions, which are described in the sections "Allocating a Linear Address Interval" and "Releasing a Linear Address Interval" later in this chapter, enlarge and shrink the address space of a process, respectively. Working at a higher level than the functions we consider here, they do not receive a memory region descriptor as their parameter, but rather the initial address, the length, and the access rights of a linear address interval.

9.3.3.1. Finding the closest region to a given address: find_vma( )

The `find_vma( )` function acts on two parameters: the address `mm` of a process memory descriptor and a linear address `addr`. It locates the first memory region whose `vm_end` field is greater than `addr` and returns the address of its descriptor; if no such region exists, it returns a `NULL` pointer. Notice that the region selected by `find_vma( )` does not necessarily include `addr` because `addr` may lie outside of any memory region.

Each memory descriptor includes an `mmap_cache` field that stores the descriptor address of the region that was last referenced by the process. This additional field is introduced to reduce the time spent in looking for the region that contains a given linear address. Locality of address references in programs makes it highly likely that if the last linear address checked belonged to a given region, the next one to be checked belongs to the same region.

The function thus starts by checking whether the region identified by `mmap_cache` includes `addr`. If so, it returns the region descriptor pointer:

```
vma = mm->mmap_cache;
if (vma && vma->vm_end > addr && vma->vm_start <= addr)
```

```
    return vma;
```

Otherwise, the memory regions of the process must be scanned, and the function looks up the memory region in the red-black tree:

```
rb_node = mm->mm_rb.rb_node;
vma = NULL;
while (rb_node) {
    vma_tmp = rb_entry(rb_node, struct vm_area_struct, vm_rb);
    if (vma_tmp->vm_end > addr) {
        vma = vma_tmp;
        if (vma_tmp->vm_start <= addr)
            break;
        rb_node = rb_node->rb_left;
    } else
        rb_node = rb_node->rb_right;
}
if (vma)
    mm->mmap_cache = vma;
return vma;
```

The function uses the `rb_entry` macro, which derives from a pointer to a node of the red-black tree the address of the corresponding memory region descriptor.

The `find_vma_prev( )` function is similar to `find_vma( )`, except that it writes in an additional `pprev` parameter a pointer to the descriptor of the memory region that precedes the one selected by the function.

Finally, the `find_vma_prepare( )` function locates the position of the new leaf in the red-black tree that corresponds to a given linear address and returns the addresses of the preceding memory region and of the parent node of the leaf to be inserted.

9.3.3.2. Finding a region that overlaps a given interval: find_vma_intersection( )

The `find_vma_intersection( )` function finds the first memory region that overlaps a given linear address interval; the `mm` parameter points to the memory descriptor of the process, while the `start_addr` and `end_addr` linear addresses specify the interval:

```
vma = find_vma(mm,start_addr);
if (vma && end_addr <= vma->vm_start)
    vma = NULL;
return vma;
```

The function returns a `NULL` pointer if no such region exists. To be exact, if `find_vma( )` returns a valid address but the memory region found starts after the end of the linear address interval, `vma` is set to `NULL`.

9.3.3.3. Finding a free interval: get_unmapped_area( )

The `get_unmapped_area( )` function searches the process address space to find an available linear address interval. The `len` parameter specifies the interval length, while a non-null `addr` parameter specifies the address from which the search must be started. If the search is successful, the function returns the initial address of the new interval; otherwise, it returns the error code `-ENOMEM`.

If the `addr` parameter is not `NULL`, the function checks that the specified address is in the User Mode address space and that it is aligned to a page boundary. Next, the function invokes either one of two methods, depending on whether the linear address interval should be used for a file memory mapping or for an anonymous memory mapping. In the former case, the function executes the `get_unmapped_area` file operation; this is discussed in Chapter 16.

In the latter case, the function executes the `get_unmapped_area` method of the memory descriptor. In turn, this method is implemented by either the `arch_get_unmapped_area( )` function, or the `arch_get_unmapped_area_topdown( )` function, according to the memory region layout of the process. As we'll see in the section "Program Segments and Process Memory Regions" in Chapter 20, every process can have two different layouts for the memory regions allocated through the `mmap( )` system call: either they start from the linear address `0x40000000` and grow towards higher addresses, or they start right above the User Mode stack and grow towards lower addresses.

Let us discuss the `arch_get_unmapped_area( )` function, which is used when the memory regions are allocated moving from lower addresses to higher ones. It is essentially equivalent to the following code fragment:

```
if (len > TASK_SIZE)
    return -ENOMEM;
addr = (addr + 0xfff) & 0xfffff000;
if (addr && addr + len <= TASK_SIZE) {
    vma = find_vma(current->mm, addr);
    if (!vma || addr + len <= vma->vm_start)
        return addr;
}
start_addr = addr = mm->free_area_cache;
for (vma = find_vma(current->mm, addr); ; vma = vma->vm_next) {
    if (addr + len > TASK_SIZE) {
        if (start_addr == (TASK_SIZE/3+0xfff)&0xfffff000)
            return -ENOMEM;
        start_addr = addr = (TASK_SIZE/3+0xfff)&0xfffff000;
        vma = find_vma(current->mm, addr);
    }
    if (!vma || addr + len <= vma->vm_start) {
        mm->free_area_cache = addr + len;
        return addr;
    }
    addr = vma->vm_end;
}
```

The function starts by checking to make sure the interval length is within `TASK_SIZE`, the limit imposed on User Mode linear addresses (usually 3 GB). If `addr` is different from zero, the function tries to allocate the interval starting from `addr`. To be on the safe side, the function rounds up the value of `addr` to a multiple of 4 KB.

If `addr` is 0 or the previous search failed, the `arch_get_unmapped_area( )` function scans the User Mode linear address space looking for a range of linear addresses not included in any memory region and

large enough to contain the new region. To speed up the search, the search's starting point is usually set to the linear address following the last allocated memory region. The mm->free_area_cache field of the memory descriptor is initialized to one-third of the User Mode linear address spaceusually, 1 GBand then updated as new memory regions are created. If the function fails in finding a suitable range of linear addresses, the search restarts from the beginningthat is, from one-third of the User Mode linear address space: in fact, the first third of the User Mode linear address space is reserved for memory regions having a predefined starting linear address, typically the text, data, and bss segments of an executable file (see Chapter 20).

The function invokes find_vma( ) to locate the first memory region ending after the search's starting point, then repeatedly considers all the following memory regions. Three cases may occur:

- The requested interval is larger than the portion of linear address space yet to be scanned (addr + len > TASK_SIZE): in this case, the function either restarts from one-third of the User Mode address space or, if the second search has already been done, returns -ENOMEM (there are not enough linear addresses to satisfy the request).
- The hole following the last scanned region is not large enough (vma != NULL && vma->vm_start < addr + len). In this case, the function considers the next region.
- If neither one of the preceding conditions holds, a large enough hole has been found. In this case, the function returns addr.

9.3.3.4. Inserting a region in the memory descriptor list: insert_vm_struct( )

insert_vm_struct( ) inserts a vm_area_struct structure in the memory region object list and red-black tree of a memory descriptor. It uses two parameters: mm, which specifies the address of a process memory descriptor, and vma, which specifies the address of the vm_area_struct object to be inserted. The vm_start and vm_end fields of the memory region object must have already been initialized. The function invokes the find_vma_prepare( ) function to look up the position in the red-black tree mm->mm_rb where vma should go. Then insert_vm_struct( ) invokes the vma_link( ) function, which in turn:

1. Inserts the memory region in the linked list referenced by mm->mmap.
2. Inserts the memory region in the red-black tree mm->mm_rb.
3. If the memory region is anonymous, inserts the region in the list headed at the corresponding anon_vma data structure (see the section "Reverse Mapping for Anonymous Pages" in Chapter 17).
4. Increases the mm->map_count counter.

If the region contains a memory-mapped file, the vma_link( ) function performs additional tasks that are described in Chapter 17.

The _ _vma_unlink( ) function receives as its parameters a memory descriptor address mm and two memory region object addresses vma and prev. Both memory regions should belong to mm, and prev should precede vma in the memory region ordering. The function removes vma from the linked list and the red-black tree of the memory descriptor. It also updates mm->mmap_cache, which stores the last referenced memory region, if this field points to the memory region just deleted.

9.3.4. Allocating a Linear Address Interval

Now let's discuss how new linear address intervals are allocated. To do this, the `do_mmap( )` function creates and initializes a new memory region for the `current` process. However, after a successful allocation, the memory region could be merged with other memory regions defined for the process.

The function uses the following parameters:

file and offset
: File object pointer `file` and file offset `offset` are used if the new memory region will map a file into memory. This topic is discussed in Chapter 16. In this section, we assume that no memory mapping is required and that `file` and `offset` are both `NULL`.

addr
: This linear address specifies where the search for a free interval must start.

len
: The length of the linear address interval.

prot
: This parameter specifies the access rights of the pages included in the memory region. Possible flags are `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`, and `PROT_NONE`. The first three flags mean the same things as the `VM_READ`, `VM_WRITE`, and `VM_EXEC` flags. `PROT_NONE` indicates that the process has none of those access rights.

flag
: This parameter specifies the remaining memory region flags:

  MAP_GROWSDOWN, MAP_LOCKED, MAP_DENYWRITE, and MAP_EXECUTABLE
  : Their meanings are identical to those of the flags listed in Table 9-5.

  MAP_SHARED and MAP_PRIVATE
  : The former flag specifies that the pages in the memory region can be shared among several processes; the latter flag has the opposite effect. Both flags refer to the `VM_SHARED` flag in the `vm_area_struct` descriptor.

  MAP_FIXED
  : The initial linear address of the interval must be exactly the one specified in the `addr` parameter.

  MAP_ANONYMOUS
  : No file is associated with the memory region (see Chapter 16).

  MAP_NORESERVE

The function doesn't have to do a preliminary check on the number of free page frames.

MAP_POPULATE
: The function should pre-allocate the page frames required for the mapping established by the memory region. This flag is significant only for memory regions that map files (see Chapter 16) and for IPC shared memory regions (see Chapter 19).

MAP_NONBLOCK
: Significant only when the MAP_POPULATE flag is set: when pre-allocating the page frames, the function must not block.

The do_mmap( ) function performs some preliminary checks on the value of offset and then executes the do_mmap_pgoff( ) function. In this chapter we will suppose that the new interval of linear address does not map a file on diskfile memory mapping is discussed in detail in Chapter 16. Here is a description of the do_mmap_pgoff( ) function for anonymous memory regions:

1. Checks whether the parameter values are correct and whether the request can be satisfied. In particular, it checks for the following conditions that prevent it from satisfying the request:
   - The linear address interval has zero length or includes addresses greater than TASK_SIZE.
   - The process has already mapped too many memory regionsthat is, the value of the map_count field of its mm memory descriptor exceeds the allowed maximum value.
   - The flag parameter specifies that the pages of the new linear address interval must be locked in RAM, but the process is not allowed to create locked memory regions, or the number of pages locked by the process exceeds the threshold stored in the signal->rlim[RLIMIT_MEMLOCK].rlim_cur field of the process descriptor.

   If any of the preceding conditions holds, do_mmap_pgoff( ) terminates by returning a negative value. If the linear address interval has a zero length, the function returns without performing any action.
2. Invokes get_unmapped_area( ) to obtain a linear address interval for the new region (see the previous section "Memory Region Handling").
3. Computes the flags of the new memory region by combining the values stored in the prot and flags parameters:

```
vm_flags = calc_vm_prot_bits(prot,flags) |
           calc_vm_flag_bits(prot,flags) |
           mm->def_flags | VM_MAYREAD | VM_MAYWRITE | VM_MAYEXEC;
if (flags & MAP_SHARED)
    vm_flags |= VM_SHARED | VM_MAYSHARE;
```

The calc_vm_prot_bits( ) function sets the VM_READ, VM_WRITE, and VM_EXEC flags in vm_flags only if the corresponding PROT_READ, PROT_WRITE, and PROT_EXEC flags in prot are set. The calc_vm_flag_bits( ) function sets the VM_GROWSDOWN, VM_DENYWRITE, VM_EXECUTABLE, and VM_LOCKED flags in vm_flags only if the corresponding MAP_GROWSDOWN, MAP_DENYWRITE, MAP_EXECUTABLE, and MAP_LOCKED flags in flags are set. A few other flags are set in vm_flags: VM_MAYREAD, VM_MAYWRITE, VM_MAYEXEC, the default flags for all memory regions in mm->def_flags,[*] and both VM_SHARED and VM_MAYSHARE if the pages of the memory region have to be shared with other processes.

[*] Actually, the def_flags field of the memory descriptor is modified only by the mlockall( ) system call, which can be used to set the VM_LOCKED flag, thus

locking all future pages of the calling process in RAM.

4. Invokes `find_vma_prepare( )` to locate the object of the memory region that shall precede the new interval, as well as the position of the new region in the red-black tree:

```
for (;;) {
    vma = find_vma_prepare(mm, addr, &prev, &rb_link, &rb_parent);
    if (!vma || vma->vm_start >= addr + len)
        break;
    if (do_munmap(mm, addr, len))
        return -ENOMEM;
}
```

   The `find_vma_prepare( )` function also checks whether a memory region that overlaps the new interval already exists. This occurs when the function returns a non-`NULL` address pointing to a region that starts before the end of the new interval. In this case, `do_mmap_pgoff( )` invokes `do_munmap( )` to remove the new interval and then repeats the whole step (see the later section "Releasing a Linear Address Interval").
5. Checks whether inserting the new memory region causes the size of the process address space `(mm->total_vm<<PAGE_SHIFT)+len` to exceed the threshold stored in the `signal->rlim[RLIMIT_AS].rlim_cur` field of the process descriptor. If so, it returns the error code `-ENOMEM`. Notice that the check is done here and not in step 1 with the other checks, because some memory regions could have been removed in step 4.
6. Returns the error code `-ENOMEM` if the `MAP_NORESERVE` flag was not set in the `flags` parameter, the new memory region contains private writable pages, and there are not enough free page frames; this last check is performed by the `security_vm_enough_memory( )` function.
7. If the new interval is private (`VM_SHARED` not set) and it does not map a file on disk, it invokes `vma_merge( )` to check whether the preceding memory region can be expanded in such a way to include the new interval. Of course, the preceding memory region must have exactly the same flags as those memory regions stored in the `vm_flags` local variable. If the preceding memory region can be expanded, `vma_merge( )` also tries to merge it with the following memory region (this occurs when the new interval fills the hole between two memory regions and all three have the same flags). In case it succeeds in expanding the preceding memory region, the function jumps to step 12.
8. Allocates a `vm_area_struct` data structure for the new memory region by invoking the `kmem_cache_alloc( )` slab allocator function.
9. Initializes the new memory region object (pointed to by `vma`):

```
vma->vm_mm = mm;
vma->vm_start = addr;
vma->vm_end = addr + len;
vma->vm_flags = vm_flags;
vma->vm_page_prot = protection_map[vm_flags & 0x0f];
vma->vm_ops = NULL;
vma->vm_pgoff = pgoff;
vma->vm_file = NULL;
vma->vm_private_data = NULL;
vma->vm_next = NULL;
INIT_LIST_HEAD(&vma->shared);
```

10. If the `MAP_SHARED` flag is set (and the new memory region doesn't map a file on disk), the region is a shared anonymous region: invokes `shmem_zero_setup( )` to initialize it. Shared anonymous regions are mainly used for interprocess communications; see the section "IPC Shared Memory" in Chapter 19.
11. Invokes `vma_link( )` to insert the new region in the memory region list and red-black tree (see the earlier section "Memory Region Handling").

12. Increases the size of the process address space stored in the `total_vm` field of the memory descriptor.
13. If the `VM_LOCKED` flag is set, it invokes `make_pages_present( )` to allocate all the pages of the memory region in succession and lock them in RAM:

```
if (vm_flags & VM_LOCKED) {
    mm->locked_vm += len >> PAGE_SHIFT;
    make_pages_present(addr, addr + len);
}
```

The `make_pages_present( )` function, in turn, invokes `get_user_pages( )` as follows:

```
write = (vma->vm_flags & VM_WRITE) != 0;
get_user_pages(current, current->mm, addr, len, write, 0, NULL, NULL);
```

The `get_user_pages( )` function cycles through all starting linear addresses of the pages between `addr` and `addr+len`; for each of them, it invokes `follow_page( )` to check whether there is a mapping to a physical page in the `current`'s Page Tables. If no such physical page exists, `get_user_pages( )` invokes `handle_mm_fault( )`, which, as we'll see in the section "Handling a Faulty Address Inside the Address Space," allocates one page frame and sets its Page Table entry according to the `vm_flags` field of the memory region descriptor.

14. Finally, it terminates by returning the linear address of the new memory region.

9.3.5. Releasing a Linear Address Interval

When the kernel must delete a linear address interval from the address space of the current process, it uses the `do_munmap( )` function. The parameters are: the address `mm` of the process's memory descriptor, the starting address `start` of the interval, and its length `len`. The interval to be deleted does not usually correspond to a memory region; it may be included in one memory region or span two or more regions.

9.3.5.1. The do_munmap( ) function

The function goes through two main phases. In the first phase (steps 16), it scans the list of memory regions owned by the process and unlinks all regions included in the linear address interval from the process address space. In the second phase (steps 712), the function updates the process Page Tables and removes the memory regions identified in the first phase. The function makes use of the `split_vma( )` and `unmap_region( )` functions, which will be described later. `do_munmap( )` executes the following steps:

1. Performs some preliminary checks on the parameter values. If the linear address interval includes addresses greater than `TASK_SIZE`, if `start` is not a multiple of 4,096, or if the linear address interval has a zero length, the function returns the error code `-EINVAL`.
2. Locates the first memory region `mpnt` that ends after the linear address interval to be deleted (`mpnt->end > start`), if any:

```
mpnt = find_vma_prev(mm, start, &prev);
```

3. If there is no such memory region, or if the region does not overlap with the linear address interval, nothing has to be done because there is no memory region in the interval:

```
end = start + len;
if (!mpnt || mpnt->vm_start >= end)
    return 0;
```

4. If the linear address interval starts inside the `mpnt` memory region, it invokes `split_vma( )` (described below) to split the `mpnt` memory region into two smaller regions: one outside the interval and the other inside the interval:

```
if (start > mpnt->vm_start) {
    if (split_vma(mm, mpnt, start, 0))
        return -ENOMEM;
    prev = mpnt;
}
```

   The `prev` local variable, which previously stored the pointer to the memory region preceding `mpnt`, is updated so that it points to `mpnt`that is, to the new memory region lying outside the linear address interval. In this way, `prev` still points to the memory region preceding the first memory region to be removed.

5. If the linear address interval ends inside a memory region, it invokes `split_vma( )` once again to split the last overlapping memory region into two smaller regions: one inside the interval and the other outside the interval:[*]

   [*] If the linear address interval is properly contained inside a memory region, the region must be replaced by two new smaller regions. When this case occurs, step 4 and step 5 break the memory region in three smaller regions: the middle region is destroyed, while the first and the last ones will be preserved.

```
last = find_vma(mm, end);
if (last && end > last->vm_start)){
    if (split_vma(mm, last, start, end, 1))
        return -ENOMEM;
}
```

6. Updates the value of `mpnt` so that it points to the first memory region in the linear address interval. If `prev` is `NULL`that is, there is no preceding memory regionthe address of the first memory region is taken from `mm->mmap`:

```
mpnt = prev ? prev->vm_next : mm->mmap;
```

7. Invokes `detach_vmas_to_be_unmapped( )` to remove the memory regions included in the linear address interval from the process's linear address space. This function essentially executes the following code:

```
vma = mpnt;
insertion_point = (prev ? &prev->vm_next : &mm->mmap);
do {
    rb_erase(&vma->vm_rb, &mm->mm_rb);
    mm->map_count--;
    tail_vma = vma;
    vma = vma->next;
} while (vma && vma->start < end);
```

```
*insertion_point = vma;
tail_vma->vm_next = NULL;
mm->map_cache = NULL;
```

The descriptors of the regions to be removed are stored in an ordered list, whose head is pointed to by the `mpnt` local variable (actually, this list is just a fragment of the original process's list of memory regions).

8. Gets the `mm->page_table_lock` spin lock.
9. Invokes `unmap_region( )` to clear the Page Table entries covering the linear address interval and to free the corresponding page frames (discussed later):

```
unmap_region(mm, mpnt, prev, start, end);
```

10. Releases the `mm->page_table_lock` spin lock.
11. Releases the descriptors of the memory regions collected in the list built in step 7:

```
do {
    struct vm_area_struct * next = mpnt->vm_next;
    unmap_vma(mm, mpnt);
    mpnt = next;
} while (mpnt != NULL);
```

The `unmap_vma( )` function is invoked on every memory region in the list; it essentially executes the following steps:

a. Updates the `mm->total_vm` and `mm->locked_vm` fields.
b. Executes the `mm->unmap_area` method of the memory descriptor. This method is implemented either by `arch_unmap_area( )` or by `arch_unmap_area_topdown( )`, according to the memory region layout of the process (see the earlier section "Memory Region Handling"). In both cases, the `mm->free_area_cache` field is updated, if needed.
c. Invokes the `close` method of the memory region, if defined.
d. If the memory region is anonymous, the function removes it from the anonymous memory region list headed at `mm->anon_vma`.
e. Invokes `kmem_cache_free( )` to release the memory region descriptor.

12. Returns 0 (success).

9.3.5.2. The split_vma( ) function

The purpose of the `split_vma( )` function is to split a memory region that intersects a linear address interval into two smaller regions, one outside of the interval and the other inside. The function receives four parameters: a memory descriptor pointer `mm`, a memory area descriptor pointer `vma` that identifies the region to be split, an address `addr` that specifies the intersection point between the interval and the memory region, and a flag `new_below` that specifies whether the intersection occurs at the beginning or at the end of the interval. The function performs the following basic steps:

1. Invokes `kmem_cache_alloc( )` to get an additional `vm_area_struct` descriptor, and stores its address in the `new` local variable. If no free memory is available, it returns `-ENOMEM`.
2. Initializes the fields of the `new` descriptor with the contents of the fields of the `vma` descriptor.

3. If the new_below flag is 0, the linear address interval starts inside the vma region, so the new region must be placed after the vma region. Thus, the function sets both the new->vm_start and the vma->vm_end fields to addr.
4. Conversely, if the new_below flag is equal to 1, the linear address interval ends inside the vma region, so the new region must be placed before the vma region. Thus, the function sets both the new->vm_end and the vma->vm_start fields to addr.
5. If the open method of the new memory region is defined, the function executes it.
6. Links the new memory region descriptor to the mm->mmap list of memory regions and to the mm->mm_rb red-black tree. Moreover, the function adjusts the red-black tree to take care of the new size of the memory region vma.
7. Returns 0 (success).

9.3.5.3. The unmap_region( ) function

The unmap_region( ) function walks through a list of memory regions and releases the page frames belonging to them. It acts on five parameters: a memory descriptor pointer mm, a pointer vma to the descriptor of the first memory region being removed, a pointer prev to the memory region preceding vma in the process's list (see steps 2 and 4 in do_munmap()), and two addresses start and end that delimit the linear address interval being removed. The function essentially executes the following steps:

1. Invokes lru_add_drain( ) (see Chapter 17).
2. Invokes the tlb_gather_mmu( ) function to initialize a per-CPU variable named mmu_gathers. The contents of mmu_gathers are architecture-dependent: generally speaking, the variable should store all information required for a successful updating of the page table entries of a process. In the 80 x 86 architecture, the tlb_gather_mmu( ) function simply saves the value of the mm memory descriptor pointer in the mmu_gathers variable of the local CPU.
3. Stores the address of the mmu_gathers variable in the tlb local variable.
4. Invokes unmap_vmas( ) to scan all Page Table entries belonging to the linear address interval: if only one CPU is available, the function invokes free_swap_and_cache( ) repeatedly to release the corresponding pages (see Chapter 17); otherwise, the function saves the pointers of the corresponding page descriptors in the mmu_gathers local variable.
5. Invokes free_pgtables(tlb,prev,start,end) to try to reclaim the Page Tables of the process that have been emptied in the previous step.
6. Invokes tlb_finish_mmu(tlb,start,end) to finish the work: in turn, this function:

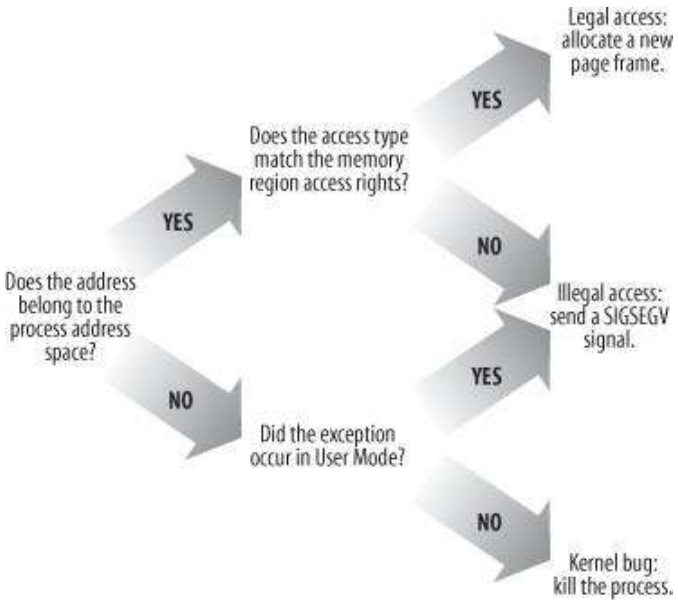
    a. Invokes flush_tlb_mm( ) to flush the TLB (see the section "Handling the Hardware Cache and the TLB" in Chapter 2).
    b. In multiprocessor system, invokes free_pages_and_swap_cache( ) to release the page frames whose pointers have been collected in the mmu_gather data structure. This function is described in Chapter 17.

9.4. Page Fault Exception Handler

As stated previously, the Linux Page Fault exception handler must distinguish exceptions caused by programming errors from those caused by a reference to a page that legitimately belongs to the process address space but simply hasn't been allocated yet.

The memory region descriptors allow the exception handler to perform its job quite efficiently. The `do_page_fault( )` function, which is the Page Fault interrupt service routine for the 80 x 86 architecture, compares the linear address that caused the Page Fault against the memory regions of the `current` process; it can thus determine the proper way to handle the exception according to the scheme that is illustrated in Figure 9-4.

Figure 9-4. Overall scheme for the Page Fault handler



In practice, things are a lot more complex because the Page Fault handler must recognize several particular subcases that fit awkwardly into the overall scheme, and it must distinguish several kinds of legal access. A detailed flow diagram of the handler is illustrated in Figure 9-5.

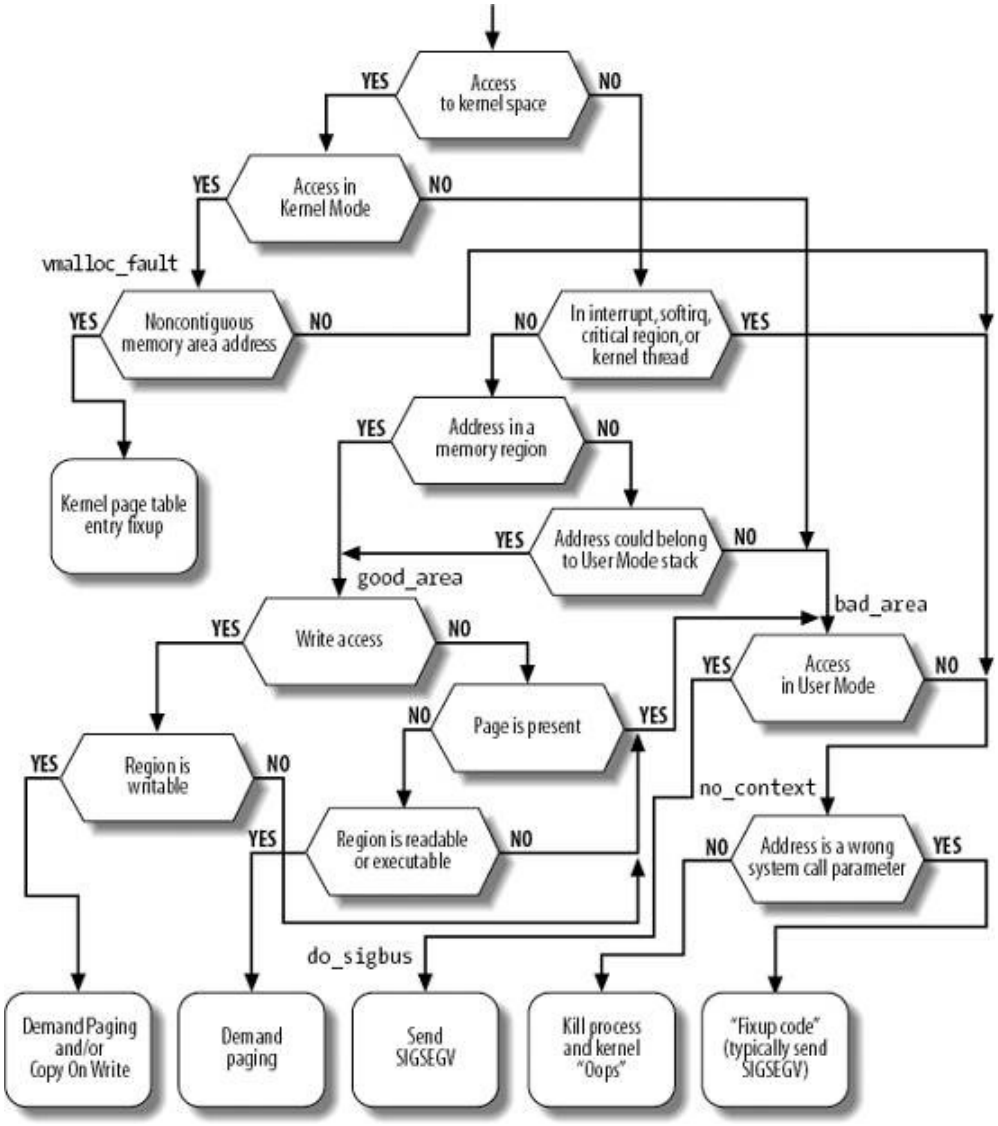
The identifiers `vmalloc_fault`, `good_area`, `bad_area`, and `no_context` are labels appearing in `do_page_fault( )` that should help you to relate the blocks of the flow diagram to specific lines of code.

The `do_ page_fault( )` function accepts the following input parameters:

- The `regs` address of a `pt_regs` structure containing the values of the microprocessor registers when the exception occurred.
- A 3-bit `error_code`, which is pushed on the stack by the control unit when the exception occurred (see "Hardware Handling of Interrupts and Exceptions" in Chapter 4). The bits have the following meanings:

- If bit 0 is clear, the exception was caused by an access to a page that is not present (the `Present` flag in the Page Table entry is clear); otherwise, if bit 0 is set, the exception was caused by an invalid access right.

Figure 9-5. The flow diagram of the Page Fault handler



- If bit 1 is clear, the exception was caused by a read or execute access; if set, the exception was caused by a write access.
- If bit 2 is clear, the exception occurred while the processor was in Kernel Mode; otherwise, it occurred in User Mode.

The first operation of `do_ page_fault( )` consists of reading the linear address that caused the Page Fault. When the exception occurs, the CPU control unit stores that value in the `cr2` control register:

```
asm("movl %%cr2,%0":"=r" (address));
if (regs->eflags & 0x00020200)
    local_irq_enable( );
tsk = current;
```

The linear address is saved in the address local variable. The function also ensures that local interrupts are enabled if they were enabled before the fault or the CPU was running in virtual-8086 mode, and saves the pointers to the process descriptor of current in the tsk local variable.

As shown at the top of Figure 9-5, do_ page_fault( ) checks whether the faulty linear address belongs to the fourth gigabyte:

```
info.si_code = SEGV_MAPERR;
if (address >= TASK_SIZE ) {
    if  (!(error_code & 0x101))
        goto vmalloc_fault;
    goto bad_area_nosemaphore;
}
```

If the exception was caused by the kernel trying to access a nonexisting page frame, a jump is made to the code at label vmalloc_fault, which takes care of faults that were likely caused by accessing a noncontiguous memory area in Kernel Mode; we describe this case in the later section "Handling Noncontiguous Memory Area Accesses." Otherwise, a jump is made to the code at the bad_area_nosemaphore label, described in the later section "Handling a Faulty Address Outside the Address Space."

Next, the handler checks whether the exception occurred while the kernel was executing some critical routine or running a kernel thread (remember that the mm field of the process descriptor is always NULL for kernel threads):

```
if (in_atomic( ) || !tsk->mm)
    goto bad_area_nosemaphore;
```

The in_atomic( ) macro yields the value one if the fault occurred while either one of the following conditions holds:

- The kernel was executing an interrupt handler or a deferrable function.
- The kernel was executing a critical region with kernel preemption disabled (see the section "Kernel Preemption" in Chapter 5).

If the Page Fault did occur in an interrupt handler, in a deferrable function, in a critical region, or in a kernel thread, do_ page_fault( ) does not try to compare the linear address with the memory regions of current. Kernel threads never use linear addresses below TASK_SIZE. Similarly, interrupt handlers, deferrable functions, and code of critical regions should not use linear addresses below TASK_SIZE because this might block the current process. (See the section "Handling a Faulty Address Outside the Address Space" later in this chapter for information on the info local variable and a description of the code at the bad_area_nosemaphore label.)

Let's suppose that the Page Fault did not occur in an interrupt handler, in a deferrable function, in a critical region, or in a kernel thread. Then the function must inspect the memory regions owned by the process to determine whether the faulty linear address is included in the process address space. In order to this, it must acquire the mmap_sem read/write semaphore of the process:

```
if (!down_read_trylock(&tsk->mm>mmap_sem)) {
    if ((error_code & 4) == 0 &&
        !search_exception_table(regs->eip))
        goto bad_area_nosemaphore;
    down_read(&tsk->mm->mmap_sem);
```

```
}
```

If kernel bugs and hardware malfunctioning can be ruled out, the current process has not already acquired the mmap_sem semaphore for writing when the Page Fault occurs. However, do_page_fault( ) wants to be sure that this is actually true, because otherwise a deadlock would occur. For that reason, the function makes use of down_read_trylock( ) instead of down_read( ) (see the section "Read/Write Semaphores" in Chapter 5). If the semaphore is closed and the Page Fault occurred in Kernel Mode, do_page_fault( ) determines whether the exception occurred while using some linear address that has been passed to the kernel as a parameter of a system call (see the next section "Handling a Faulty Address Outside the Address Space"). In this case, do_page_fault( ) knows for sure that the semaphore is owned by another processbecause every system call service routine carefully avoids acquiring the mmap_sem semaphore for writing before accessing the User Mode address spaceso the function waits until the semaphore is released. Otherwise, the Page Fault is due to a kernel bug or to a serious hardware problem, so the function jumps to the bad_area_nosemaphore label.

Let's assume that the mmap_sem semaphore has been safely acquired for reading. Now do_page_fault( ) looks for a memory region containing the faulty linear address:

```
vma = find_vma(tsk->mm, address);
if (!vma)
    goto bad_area;
if (vma->vm_start <= address)
    goto good_area;
```

If vma is NULL, there is no memory region ending after address, and thus the faulty address is certainly bad. On the other hand, if the first memory region ending after address includes address, the function jumps to the code at label good_area.

If none of the two "if" conditions are satisfied, the function has determined that address is not included in any memory region; however, it must perform an additional check, because the faulty address may have been caused by a push or pusha instruction on the User Mode stack of the process.

Let's make a short digression to explain how stacks are mapped into memory regions. Each region that contains a stack expands toward lower addresses; its VM_GROWSDOWN flag is set, so the value of its vm_end field remains fixed while the value of its vm_start field may be decreased. The region boundaries include, but do not delimit precisely, the current size of the User Mode stack. The reasons for the fuzz factor are:

- The region size is a multiple of 4 KB (it must include complete pages) while the stack size is arbitrary.
- Page frames assigned to a region are never released until the region is deleted; in particular, the value of the vm_start field of a region that includes a stack can only decrease; it can never increase. Even if the process executes a series of pop instructions, the region size remains unchanged.

It should now be clear how a process that has filled up the last page frame allocated to its stack may cause a Page Fault exception: the push refers to an address outside of the region (and to a nonexistent page frame). Notice that this kind of exception is not caused by a programming error; thus it must be handled separately by the Page Fault handler.

We now return to the description of do_ page_fault( ), which checks for the case described previously:

```
if (!(vma->vm_flags & VM_GROWSDOWN))
```

```
    goto bad_area;
if (error_code & 4      /* User Mode */
    && address + 32 < regs->esp)
        goto bad_area;
if (expand_stack(vma, address))
    goto bad_area;
goto good_area;
```

If the VM_GROWSDOWN flag of the region is set and the exception occurred in User Mode, the function checks whether address is smaller than the regs->esp stack pointer (it should be only a little smaller). Because a few stack-related assembly language instructions (such as pusha) perform a decrement of the esp register only after the memory access, a 32-byte tolerance interval is granted to the process. If the address is high enough (within the tolerance granted), the code invokes the expand_stack( ) function to check whether the process is allowed to extend both its stack and its address space; if everything is OK, it sets the vm_start field of vma to address and returns 0; otherwise, it returns -ENOMEM.

Note that the preceding code skips the tolerance check whenever the VM_GROWSDOWN flag of the region is set and the exception did not occur in User Mode. These conditions mean that the kernel is addressing the User Mode stack and that the code should always run expand_stack( ).

9.4.1. Handling a Faulty Address Outside the Address Space

If address does not belong to the process address space, do_page_fault( ) proceeds to execute the statements at the label bad_area. If the error occurred in User Mode, it sends a SIGSEGV signal to current (see the section "Generating a Signal" in Chapter 11) and terminates:

```
bad_area:
up_read(&tsk->mm->mmap_sem);
bad_area_nosemaphore:
if (error_code & 4) {   /* User Mode */
    tsk->thread.cr2 = address;
    tsk->thread.error_code = error_code | (address >= TASK_SIZE);
    tsk->thread.trap_no = 14;
    info.si_signo = SIGSEGV;
    info.si_errno = 0;
    info.si_addr = (void *) address;
    force_sig_info(SIGSEGV, &info, tsk);
    return;
}
```

The force_sig_info( ) function makes sure that the process does not ignore or block the SIGSEGV signal, and sends the signal to the User Mode process while passing some additional information in the info local variable (see the section "Generating a Signal" in Chapter 11). The info.si_code field is already set to SEGV_MAPERR (if the exception was due to a nonexisting page frame) or to SEGV_ACCERR (if the exception was due to an invalid access to an existing page frame).

If the exception occurred in Kernel Mode (bit 2 of error_code is clear), there are still two alternatives:

- The exception occurred while using some linear address that has been passed to the kernel as a parameter of a system call.
- The exception is due to a real kernel bug.

The function distinguishes these two alternatives as follows:

```
no_context:
if ((fixup = search_exception_table(regs->eip)) != 0) {
    regs->eip = fixup;
    return;
}
```

In the first case, it jumps to a "fixup code," which typically sends a `SIGSEGV` signal to `current` or terminates a system call handler with a proper error code (see the section "Dynamic Address Checking: The Fix-up Code" in Chapter 10).

In the second case, the function prints a complete dump of the CPU registers and of the Kernel Mode stack both on the console and on a system message buffer; it then kills the current process by invoking the `do_exit( )` function (see Chapter 20). This is the so-called "Kernel oops" error, named after the message displayed. The dumped values can be used by kernel hackers to reconstruct the conditions that triggered the bug, and thus find and correct it.

9.4.2. Handling a Faulty Address Inside the Address Space

If `address` belongs to the process address space, `do_ page_fault( )` proceeds to the statement labeled `good_area`:

```
good_area:
info.si_code = SEGV_ACCERR;
write = 0;
if (error_code & 2) { /* write access */
    if (!(vma->vm_flags & VM_WRITE))
        goto bad_area;
    write++;
} else                /* read access */
    if ((error_code & 1) || !(vma->vm_flags & (VM_READ | VM_EXEC)))
        goto bad_area;
```

If the exception was caused by a write access, the function checks whether the memory region is writable. If not, it jumps to the `bad_area` code; if so, it sets the `write` local variable to 1.

If the exception was caused by a read or execute access, the function checks whether the page is already present in RAM. In this case, the exception occurred because the process tried to access a privileged page frame (one whose `User/Supervisor` flag is clear) in User Mode, so the function jumps to the `bad_area` code.[*] If the page is not present, the function also checks whether the memory region is readable or executable.

> [*] However, this case should never happen, because the kernel does not assign privileged page frames to the processes.

If the memory region access rights match the access type that caused the exception, the `handle_mm_fault( )` function is invoked to allocate a new page frame:

```
survive:
ret = handle_mm_fault(tsk->mm, vma, address, write);
```

```
if (ret == VM_FAULT_MINOR || ret == VM_FAULT_MAJOR) {
    if (ret == VM_FAULT_MINOR) tsk->min_flt++; else tsk->maj_flt++;
    up_read(&tsk->mm->mmap_sem);
    return;
}
```

The `handle_mm_fault( )` function returns `VM_FAULT_MINOR` or `VM_FAULT_MAJOR` if it succeeded in allocating a new page frame for the process. The value `VM_FAULT_MINOR` indicates that the Page Fault has been handled without blocking the current process; this kind of Page Fault is called minor fault. The value `VM_FAULT_MAJOR` indicates that the Page Fault forced the current process to sleep (most likely because time was spent while filling the page frame assigned to the process with data read from disk); a Page Fault that blocks the current process is called a major fault. The function can also return `VM_FAULT_OOM` (for not enough memory) or `VM_FAULT_SIGBUS` (for every other error).

If `handle_mm_fault( )` returns the value `VM_FAULT_SIGBUS`, a `SIGBUS` signal is sent to the process:

```
if (ret == VM_FAULT_SIGBUS) {
do_sigbus:
    up_read(&tsk->mm->mmap_sem);
    if (!(error_code & 4)) /* Kernel Mode */
        goto no_context;
    tsk->thread.cr2 = address;
    tsk->thread.error_code = error_code;
    tsk->thread.trap_no = 14;
    info.si_signo = SIGBUS;
    info.si_errno = 0;
    info.si_code = BUS_ADRERR;
    info.si_addr = (void *) address;
    force_sig_info(SIGBUS, &info, tsk);
}
```

If `handle_mm_fault( )` cannot allocate the new page frame, it returns the value `VM_FAULT_OOM`; in this case, the kernel usually kills the current process. However, if `current` is the init process, it is just put at the end of the run queue and the scheduler is invoked; once `init` resumes its execution, `handle_mm_fault( )` is executed again:

```
if (ret == VM_FAULT_OOM) {
  out_of_memory:
    up_read(&tsk->mm->mmap_sem);
    if (tsk->pid != 1) {
        if (error_code & 4) /* User Mode */
            do_exit(SIGKILL);
        goto no_context;
    }
    yield();
    down_read(&tsk->mm->mmap_sem);
    goto survive;
}
```

The `handle_mm_fault( )` function acts on four parameters:

mm

A pointer to the memory descriptor of the process that was running on the CPU when the exception occurred

vma
: A pointer to the descriptor of the memory region, including the linear address that caused the exception

address
: The linear address that caused the exception

write_access
: Set to 1 if `tsk` attempted to write in `address` and to 0 if `tsk` attempted to read or execute it

The function starts by checking whether the Page Middle Directory and the Page Table used to map `address` exist. Even if `address` belongs to the process address space, the corresponding Page Tables might not have been allocated, so the task of allocating them precedes everything else:

```
pgd = pgd_offset(mm, address);
spin_lock(&mm->page_table_lock);
pud = pud_alloc(mm, pgd, address);
if (pud) {
    pmd = pmd_alloc(mm, pud, address);
    if (pmd) {
        pte = pte_alloc_map(mm, pmd, address);
        if (pte)
            return handle_pte_fault(mm, vma, address,
                                    write_access, pte, pmd);
    }
}
spin_unlock(&mm->page_table_lock);
return VM_FAULT_OOM;
```

The `pgd` local variable contains the Page Global Directory entry that refers to `address`; `pud_alloc( )` and `pmd_alloc( )` are invoked to allocate, if needed, a new Page Upper Directory and a new Page Middle Directory, respectively.[*] `pte_alloc_map( )` is then invoked to allocate, if needed, a new Page Table. If both operations are successful, the `pte` local variable points to the Page Table entry that refers to `address`. The `handle_pte_fault( )` function is then invoked to inspect the Page Table entry corresponding to `address` and to determine how to allocate a new page frame for the process:

[*] On 80 x 86 microprocessors, these allocations never occur, because the Page Upper Directories are always included in the Page Global Directory, and the Page Middle Directories are either included in the Page Upper Directory (PAE not enabled) or allocated together with the Page Upper Directory (PAE enabled).

- If the accessed page is not presentthat is, if it is not already stored in any page framethe kernel allocates a new page frame and initializes it properly; this technique is called demand paging .
- If the accessed page is present but is marked read-onlyi.e., if it is already stored in a page framethe kernel allocates a new page frame and initializes its contents by copying the old page frame data; this technique is called Copy On Write.

9.4.3. Demand Paging

The term demand paging denotes a dynamic memory allocation technique that consists of deferring page frame allocation until the last possible momentuntil the process attempts to address a page that is not present in RAM, thus causing a Page Fault exception.

The motivation behind demand paging is that processes do not address all the addresses included in their address space right from the start; in fact, some of these addresses may never be used by the process. Moreover, the program locality principle (see the section "Hardware Cache" in Chapter 2) ensures that, at each stage of program execution, only a small subset of the process pages are really referenced, and therefore the page frames containing the temporarily useless pages can be used by other processes. Demand paging is thus preferable to global allocation (assigning all page frames to the process right from the start and leaving them in memory until program termination), because it increases the average number of free page frames in the system and therefore allows better use of the available free memory. From another viewpoint, it allows the system as a whole to get better throughput with the same amount of RAM.

The price to pay for all these good things is system overhead: each Page Fault exception induced by demand paging must be handled by the kernel, thus wasting CPU cycles. Fortunately, the locality principle ensures that once a process starts working with a group of pages, it sticks with them without addressing other pages for quite a while. Thus, Page Fault exceptions may be considered rare events.

An addressed page may not be present in main memory either because the page was never accessed by the process, or because the corresponding page frame has been reclaimed by the kernel (see Chapter 17).

In both cases, the page fault handler must assign a new page frame to the process. How this page frame is initialized, however, depends on the kind of page and on whether the page was previously accessed by the process. In particular:

1. Either the page was never accessed by the process and it does not map a disk file, or the page maps a disk file. The kernel can recognize these cases because the Page Table entry is filled with zerosi.e., the `pte_none` macro returns the value 1.
2. The page belongs to a non-linear disk file mapping (see the section "Non-Linear Memory Mappings" in Chapter 16). The kernel can recognize this case, because the `Present` flag is cleared and the `Dirty` flag is seti.e., the `pte_file` macro returns the value 1.
3. The page was already accessed by the process, but its content is temporarily saved on disk. The kernel can recognize this case because the Page Table entry is not filled with zeros, but the `Present` and `Dirty` flags are cleared.

Thus, the `handle_ pte_fault( )` function is able to distinguish the three cases by inspecting the Page Table entry that refers to `address`:

```
entry = *pte;
if (!pte_present(entry)) {
    if (pte_none(entry))
        return do_no_page(mm, vma, address, write_access, pte, pmd);
    if (pte_file(entry))
        return do_file_page(mm, vma, address, write_access, pte, pmd);
    return do_swap_page(mm, vma, address, pte, pmd, entry, write_access);
}
```

We'll examine cases 2 and 3 in Chapter 16 and in Chapter 17, respectively.

In case 1, when the page was never accessed or the page linearly maps a disk file, the do_no_page( ) function is invoked. There are two ways to load the missing page, depending on whether the page is mapped to a disk file. The function determines this by checking the nopage method of the vma memory region object, which points to the function that loads the missing page from disk into RAM if the page is mapped to a file. Therefore, the possibilities are:

- The vma->vm_ops->nopage field is not NULL. In this case, the memory region maps a disk file and the field points to the function that loads the page. This case is covered in the section "Demand Paging for Memory Mapping" in Chapter 16 and in the section "IPC Shared Memory" in Chapter 19.
- Either the vma->vm_ops field or the vma->vm_ops->nopage field is NULL. In this case, the memory region does not map a file on diski.e., it is an anonymous mapping . Thus, do_no_ page( ) invokes the do_anonymous_page( ) function to get a new page frame:

```
if (!vma->vm_ops || !vma->vm_ops->nopage)
    return do_anonymous_page(mm, vma, page_table, pmd,
                             write_access, address);
```

The do_anonymous_page( ) function[*] handles write and read requests separately:

[*] To simplify the description of this function, we skip the statements that deal with reverse mapping, a topic that will be covered in the section "Reverse Mapping" in Chapter 17.

```
if (write_access) {
    pte_unmap(page_table);
    spin_unlock(&mm->page_table_lock);
    page = alloc_page(GFP_HIGHUSER | _ _GFP_ZERO);
    spin_lock(&mm->page_table_lock);
    page_table = pte_offset_map(pmd, addr);
    mm->rss++;
    entry = maybe_mkwrite(pte_mkdirty(mk_pte(page,
                                             vma->vm_page_prot)), vma);
    lru_cache_add_active(page);
    SetPageReferenced(page);
    set_pte(page_table, entry);
    pte_unmap(page_table);
    spin_unlock(&mm->page_table_lock);
    return VM_FAULT_MINOR;
}
```

The first execution of the pte_unmap macro releases the temporary kernel mapping for the high-memory physical address of the Page Table entry established by pte_offset_map before invoking the handle_pte_fault( ) function (see Table 2-7 in the section "Page Table Handling" in Chapter 2). The following pair or pte_offset_map and pte_unmap macros acquires and releases the same temporary kernel mapping. The temporary kernel mapping has to be released before invoking alloc_page( ), because this function might block the current process.

The function increases the rss field of the memory descriptor to keep track of the number of page frames allocated to the process. The Page Table entry is then set to the physical address of the page frame, which is marked as writable[†] and dirty. The lru_cache_add_active( ) function inserts the new page frame in the swap-related data structures; we discuss it in Chapter 17.

[†] If a debugger attempts to write in a page belonging to a read-only memory region of the traced process, the kernel does not set the Read/Write flag. The maybe_mkwrite( ) function takes care of this special case.

Conversely, when handling a read access, the content of the page is irrelevant because the process is addressing it for the first time. It is safer to give a page filled with zeros to the process rather than an old page filled with information written by some other process. Linux goes one step further in the spirit of demand paging. There is no need to assign a new page frame filled with zeros to the process right away, because we might as well give it an existing page called zero page , thus deferring further page frame allocation. The zero page is allocated statically during kernel initialization in the `empty_zero_page` variable (an array of 4,096 bytes filled with zeros).

The Page Table entry is thus set with the physical address of the zero page:

```
entry = pte_wrprotect(mk_pte(virt_to_page(empty_zero_page),
                             vma->vm_page_prot));
set_pte(page_table, entry);
spin_unlock(&mm->page_table_lock);
return VM_FAULT_MINOR;
```

Because the page is marked as nonwritable, if the process attempts to write in it, the Copy On Write mechanism is activated. Only then does the process get a page of its own to write in. The mechanism is described in the next section.

9.4.4. Copy On Write

First-generation Unix systems implemented process creation in a rather clumsy way: when a `fork( )` system call was issued, the kernel duplicated the whole parent address space in the literal sense of the word and assigned the copy to the child process. This activity was quite time consuming since it required:

- Allocating page frames for the Page Tables of the child process
- Allocating page frames for the pages of the child process
- Initializing the Page Tables of the child process
- Copying the pages of the parent process into the corresponding pages of the child process

This way of creating an address space involved many memory accesses, used up many CPU cycles, and completely spoiled the cache contents. Last but not least, it was often pointless because many child processes start their execution by loading a new program, thus discarding entirely the inherited address space (see Chapter 20).

Modern Unix kernels, including Linux, follow a more efficient approach called Copy On Write (COW). The idea is quite simple: instead of duplicating page frames, they are shared between the parent and the child process. However, as long as they are shared, they cannot be modified. Whenever the parent or the child process attempts to write into a shared page frame, an exception occurs. At this point, the kernel duplicates the page into a new page frame that it marks as writable. The original page frame remains write-protected: when the other process tries to write into it, the kernel checks whether the writing process is the only owner of the page frame; in such a case, it makes the page frame writable for the process.

The `_count` field of the page descriptor is used to keep track of the number of processes that are sharing the corresponding page frame. Whenever a process releases a page frame or a Copy On Write is executed on it, its `_count` field is decreased; the page frame is freed only when `_count` becomes –1 (see the section "Page Descriptors" in Chapter 8).

Let's now describe how Linux implements COW. When `handle_ pte_fault( )` determines that the

Page Fault exception was caused by an access to a page present in memory, it executes the following instructions:

```
if (pte_present(entry)) {
    if (write_access) {
        if (!pte_write(entry))
            return do_wp_page(mm, vma, address, pte, pmd, entry);
        entry = pte_mkdirty(entry);
    }
    entry = pte_mkyoung(entry);
    set_pte(pte, entry);
    flush_tlb_page(vma, address);
    pte_unmap(pte);
    spin_unlock(&mm->page_table_lock);
    return VM_FAULT_MINOR;
}
```

The `handle_pte_fault( )` function is architecture-independent: it considers each possible violation of the page access rights. However, in the 80 x 86 architecture, if the page is present, the access was for writing and the page frame is write-protected (see the earlier section "Handling a Faulty Address Inside the Address Space"). Thus, the `do_wp_page( )` function is always invoked.

The `do_wp_page( )` function[*] starts by deriving the page descriptor of the page frame referenced by the Page Table entry involved in the Page Fault exception. Next, the function determines whether the page must really be duplicated. If only one process owns the page, Copy On Write does not apply, and the process should be free to write the page. Basically, the function reads the `_count` field of the page descriptor: if it is equal to 0 (a single owner), COW must not be done. Actually, the check is slightly more complicated, because the `_count` field is also increased when the page is inserted into the swap cache (see the section "The Swap Cache" in Chapter 17) and when the `PG_private` flag in the page descriptor is set. However, when COW is not to be done, the page frame is marked as writable, so that it does not cause further Page Fault exceptions when writes are attempted:

[*] To simplify the description of this function, we skip the statements that deal with reverse mapping, a topic that will be covered in the section "Reverse Mapping" in Chapter 17.

```
set_pte(page_table, maybe_mkwrite(pte_mkyoung(pte_mkdirty(pte)),vma));
flush_tlb_page(vma, address);
pte_unmap(page_table);
spin_unlock(&mm->page_table_lock);
return VM_FAULT_MINOR;
```

If the page is shared among several processes by means of COW, the function copies the content of the old page frame (`old_page`) into the newly allocated one (`new_page`). To avoid race conditions, `get_page( )` is invoked to increase the usage counter of `old_page` before starting the copy operation:

```
old_page = pte_page(pte);
pte_unmap(page_table);
get_page(old_page);
spin_unlock(&mm->page_table_lock);
if (old_page == virt_to_page(empty_zero_page))
    new_page = alloc_page(GFP_HIGHUSER | __GFP_ZERO);
} else {
    new_page = alloc_page(GFP_HIGHUSER);
    vfrom = kmap_atomic(old_page, KM_USER0)
    vto = kmap_atomic(new_page, KM_USER1);
    copy_page(vto, vfrom);
```

```
    kunmap_atomic(vfrom, KM_USER0);
    kunmap_atomic(vto, KM_USER0);
}
```

If the old page is the zero page, the new frame is efficiently filled with zeros when it is allocated (__GFP_ZERO flag). Otherwise, the page frame content is copied using the copy_page( ) macro. Special handling for the zero page is not strictly required, but it improves the system performance, because it preserves the microprocessor hardware cache by making fewer address references.

Because the allocation of a page frame can block the process, the function checks whether the Page Table entry has been modified since the beginning of the function (pte and *page_table do not have the same value). In this case, the new page frame is released, the usage counter of old_page is decreased (to undo the increment made previously), and the function terminates.

If everything looks OK, the physical address of the new page frame is finally written into the Page Table entry, and the corresponding TLB register is invalidated:

```
spin_lock(&mm->page_table_lock);
entry = maybe_mkwrite(pte_mkdirty(mk_pte(new_page,
                                         vma->vm_page_prot)),vma);
set_pte(page_table, entry);
flush_tlb_page(vma, address);
lru_cache_add_active(new_page);
pte_unmap(page_table);
spin_unlock(&mm->page_table_lock);
```

The lru_cache_add_active( ) function inserts the new page frame in the swap-related data structures; see Chapter 17 for its description.

Finally, do_wp_page( ) decreases the usage counter of old_page twice. The first decrement undoes the safety increment made before copying the page frame contents; the second decrement reflects the fact that the current process no longer owns the page frame.

9.4.5. Handling Noncontiguous Memory Area Accesses

We have seen in the section "Noncontiguous Memory Area Management" in Chapter 8 that the kernel is quite lazy in updating the Page Table entries corresponding to noncontiguous memory areas. In fact, the vmalloc( ) and vfree( ) functions limit themselves to updating the master kernel Page Tables (i.e., the Page Global Directory init_mm.pgd and its child Page Tables).

However, once the kernel initialization phase ends, the master kernel Page Tables are not directly used by any process or kernel thread. Thus, consider the first time that a process in Kernel Mode accesses a noncontiguous memory area. When translating the linear address into a physical address, the CPU's memory management unit encounters a null Page Table entry and raises a Page Fault. However, the handler recognizes this special case because the exception occurred in Kernel Mode, and the faulty linear address is greater than TASK_SIZE. Thus, the do_page_fault( ) handler checks the corresponding master kernel Page Table entry:

```
vmalloc_fault:
asm("movl %%cr3
```

```
,%0":"=r" (pgd_paddr));
    pgd = pgd_index(address) + (pgd_t *) _ _va(pgd_paddr);
    pgd_k = init_mm.pgd + pgd_index(address);
    if (!pgd_present(*pgd_k))
        goto no_context;
    pud = pud_offset(pgd, address);
    pud_k = pud_offset(pgd_k, address);
    if (!pud_present(*pud_k))
        goto no_context;
    pmd = pmd_offset(pud, address);
    pmd_k = pmd_offset(pud_k, address);
    if (!pmd_present(*pmd_k))
        goto no_context;
    set_pmd(pmd, *pmd_k);
    pte_k = pte_offset_kernel(pmd_k, address);
    if (!pte_present(*pte_k))
        goto no_context;
    return;
```

The `pgd_paddr` local variable is loaded with the physical address of the Page Global Directory of the current process, which is stored in the `cr3` register.[*] The `pgd` local variable is then loaded with the linear address corresponding to `pgd_paddr`, and the `pgd_k` local variable is loaded with the linear address of the master kernel Page Global Directory.

> [*] The kernel doesn't use `current->mm->pgd` to derive the address because this fault can occur anytime, even during a process switch.

If the master kernel Page Global Directory entry corresponding to the faulty linear address is null, the function jumps to the code at the `no_context` label (see the earlier section "Handling a Faulty Address Outside the Address Space"). Otherwise, the function looks at the master kernel Page Upper Directory entry and at the master kernel Page Middle Directory entry corresponding to the faulty linear address. Again, if either one of these entries is null, a jump is done to the `no_context` label. Otherwise, the master entry is copied into the corresponding entry of the process's Page Middle Directory.[*] Then the whole operation is repeated with the master Page Table entry.

> [*] You might remember from the section "Paging in Linux" in Chapter 2 that if PAE is enabled then the Page Upper Directory entry cannot be null; otherwise, if PAE is disabled, setting the Page Middle Directory entry implicitly sets the Page Upper Directory entry, too.

9.5. Creating and Deleting a Process Address Space

Of the six typical cases mentioned earlier in the section "The Process's Address Space," in which a process gets new memory regions, the first oneissuing a `fork( )` system callrequires the creation of a whole new address space for the child process. Conversely, when a process terminates, the kernel destroys its address space. In this section, we discuss how these two activities are performed by Linux.

9.5.1. Creating a Process Address Space

In the section "The clone( ), fork( ), and vfork( ) System Calls" in Chapter 3, we mentioned that the kernel invokes the `copy_mm( )` function while creating a new process. This function creates the process address space by setting up all Page Tables and memory descriptors of the new process.

Each process usually has its own address space, but lightweight processes can be created by calling `clone( )` with the `CLONE_VM` flag set. These processes share the same address space; that is, they are allowed to address the same set of pages.

Following the COW approach described earlier, traditional processes inherit the address space of their parent: pages stay shared as long as they are only read. When one of the processes attempts to write one of them, however, the page is duplicated; after some time, a forked process usually gets its own address space that is different from that of the parent process. Lightweight processes, on the other hand, use the address space of their parent process. Linux implements them simply by not duplicating address space. Lightweight processes can be created considerably faster than normal processes, and the sharing of pages can also be considered a benefit as long as the parent and children coordinate their accesses carefully.

If the new process has been created by means of the `clone( )` system call and if the `CLONE_VM` flag of the `flag` parameter is set, `copy_mm( )` gives the clone (`tsk`) the address space of its parent (`current`):

```
if (clone_flags & CLONE_VM) {
    atomic_inc(&current->mm->mm_users);
    spin_unlock_wait(&current->mm->page_table_lock);
    tsk->mm = current->mm;
    tsk->active_mm = current->mm;
    return 0;
}
```

Invoking the `spin_unlock_wait( )` function ensures that, if the page table spin lock of the process is held by some other CPU, the page fault handler does not terminate until that lock is released. In fact, beside protecting the page tables, this spin lock must forbid the creation of new lightweight processes sharing the `current->mm` descriptor.

If the `CLONE_VM` flag is not set, `copy_mm( )` must create a new address space (even though no memory is allocated within that address space until the process requests an address). The function allocates a new memory descriptor, stores its address in the `mm` field of the new process descriptor `tsk`, and copies the contents of `current->mm` into `tsk->mm`. It then changes a few fields of the new descriptor:

```
tsk->mm = kmem_cache_alloc(mm_cachep, SLAB_KERNEL);
```

```
memcpy(tsk->mm, current->mm, sizeof(*tsk->mm));
atomic_set(&tsk->mm->mm_users, 1);
atomic_set(&tsk->mm->mm_count, 1);
init_rwsem(&tsk->mm->mmap_sem);
tsk->mm->core_waiters = 0;
tsk->mm->page_table_lock = SPIN_LOCK_UNLOCKED;
tsk->mm->ioctx_list_lock = RW_LOCK_UNLOCKED;
tsk->mm->ioctx_list = NULL;
tsk->mm->default_kioctx = INIT_KIOCTX(tsk->mm->default_kioctx,
                                      *tsk->mm);
tsk->mm->free_area_cache = (TASK_SIZE/3+0xfff)&0xfffff000;
tsk->mm->pgd = pgd_alloc(tsk->mm);
tsk->mm->def_flags = 0;
```

Remember that the `pgd_alloc( )` macro allocates a Page Global Directory for the new process.

The architecture-dependent `init_new_context( )` function is then invoked: when dealing with 80 x 86 processors, this function checks whether the current process owns a customized Local Descriptor Table; if so, `init_new_context( )` makes a copy of the Local Descriptor Table of `current` and adds it to the address space of `tsk`.

Finally, the `dup_mmap( )` function is invoked to duplicate both the memory regions and the Page Tables of the parent process. This function inserts the new memory descriptor `tsk->mm` in the global list of memory descriptors. Then it scans the list of regions owned by the parent process, starting from the one pointed to by `current->mm->mmap`. It duplicates each `vm_area_struct` memory region descriptor encountered and inserts the copy in the list of regions and in the red-black tree owned by the child process.

Right after inserting a new memory region descriptor, `dup_mmap( )` invokes `copy_page_range( )` to create, if necessary, the Page Tables needed to map the group of pages included in the memory region and to initialize the new Page Table entries. In particular, each page frame corresponding to a private, writable page (`VM_SHARED` flag off and `VM_MAYWRITE` flag on) is marked as read-only for both the parent and the child, so that it will be handled with the Copy On Write mechanism.

9.5.2. Deleting a Process Address Space

When a process terminates, the kernel invokes the `exit_mm( )` function to release the address space owned by that process:

```
mm_release(tsk, tsk->mm);
if (!(mm = tsk->mm)) /* kernel thread ? */
    return;
down_read(&mm->mmap_sem);
```

The `mm_release( )` function essentially wakes up all processes sleeping in the `tsk->vfork_done` completion (see the section "Completions" in Chapter 5). Typically, the corresponding wait queue is nonempty only if the exiting process was created by means of the `vfork( )` system call (see the section "The clone( ), fork( ), and vfork( ) System Calls" in Chapter 3).

If the process being terminated is not a kernel thread, the `exit_mm( )` function must release the memory descriptor and all related data structures. First of all, it checks whether the `mm->core_waiters` flag is set: if it does, then the process is dumping the contents of the memory to a core file. To avoid corruption in the

core file, the function makes use of the `mm->core_done` and `mm->core_startup_done` completions to serialize the execution of the lightweight processes sharing the same memory descriptor `mm`.

Next, the function increases the memory descriptor's main usage counter, resets the `mm` field of the process descriptor, and puts the processor in lazy TLB mode (see "Handling the Hardware Cache and the TLB" in Chapter 2):

```
atomic_inc(&mm->mm_count);
spin_lock(tsk->alloc_lock);
tsk->mm = NULL;
up_read(&mm->map_sem);
enter_lazy_tlb(mm, current);
spin_unlock(tsk->alloc_lock);
mmput(mm);
```

Finally, the `mmput( )` function is invoked to release the Local Descriptor Table, the memory region descriptors, and the Page Tables. The memory descriptor itself, however, is not released, because `exit_mm( )` has increased the main usage counter. The descriptor will be released by the `finish_task_switch( )` function when the process being terminated will be effectively evicted from the local CPU (see the section "The schedule( ) Function" in Chapter 7).

9.6. Managing the Heap

Each Unix process owns a specific memory region called the heap, which is used to satisfy the process's dynamic memory requests. The `start_brk` and `brk` fields of the memory descriptor delimit the starting and ending addresses, respectively, of that region.

The following APIs can be used by the process to request and release dynamic memory:

malloc(size)
: Requests `size` bytes of dynamic memory; if the allocation succeeds, it returns the linear address of the first memory location.

calloc(n,size)
: Requests an array consisting of `n` elements of size `size`; if the allocation succeeds, it initializes the array components to 0 and returns the linear address of the first element.

realloc(ptr,size)
: Changes the size of a memory area previously allocated by `malloc( )` or `calloc( )`.

free(addr)
: Releases the memory region allocated by `malloc( )` or `calloc( )` that has an initial address of `addr`.

brk(addr)
: Modifies the size of the heap directly; the `addr` parameter specifies the new value of `current->mm->brk`, and the return value is the new ending address of the memory region (the process must check whether it coincides with the requested `addr` value).

sbrk(incr)
: Is similar to `brk( )`, except that the `incr` parameter specifies the increment or decrement of the heap size in bytes.

The `brk( )` function differs from the other functions listed because it is the only one implemented as a system call. All the other functions are implemented in the C library by using `brk( )` and `mmap( )`.[*]

[*] The `realloc( )` library function can also make use of the `mremap( )` system call.

When a process in User Mode invokes the `brk( )` system call, the kernel executes the `sys_brk(addr)` function. This function first verifies whether the `addr` parameter falls inside the memory region that contains the process code; if so, it returns immediately because the heap cannot overlap with memory region containing the process's code:

```
mm = current->mm;
```

```
down_write(&mm->mmap_sem);
if (addr < mm->end_code) {
out:
    up_write(&mm->mmap_sem);
    return mm->brk;
}
```

Because the `brk( )` system call acts on a memory region, it allocates and deallocates whole pages. Therefore, the function aligns the value of `addr` to a multiple of `PAGE_SIZE` and compares the result with the value of the `brk` field of the memory descriptor:

```
newbrk = (addr + 0xfff) & 0xfffff000;
oldbrk = (mm->brk + 0xfff) & 0xfffff000;
if (oldbrk == newbrk) {
    mm->brk = addr;
    goto out;
}
```

If the process asked to shrink the heap, `sys_brk( )` invokes the `do_munmap( )` function to do the job and then returns:

```
if (addr <= mm->brk) {
    if (!do_munmap(mm, newbrk, oldbrk-newbrk))
        mm->brk = addr;
    goto out;
}
```

If the process asked to enlarge the heap, `sys_brk( )` first checks whether the process is allowed to do so. If the process is trying to allocate memory outside its limit, the function simply returns the original value of `mm->brk` without allocating more memory:

```
rlim = current->signal->rlim[RLIMIT_DATA].rlim_cur;
if (rlim < RLIM_INFINITY && addr - mm->start_data > rlim)
    goto out;
```

The function then checks whether the enlarged heap would overlap some other memory region belonging to the process and, if so, returns without doing anything:

```
if (find_vma_intersection(mm, oldbrk, newbrk+PAGE_SIZE))
    goto out;
```

If everything is OK, the `do_brk( )` function is invoked. If it returns the `oldbrk` value, the allocation was successful and `sys_brk( )` returns the value `addr`; otherwise, it returns the old `mm->brk` value:

```
if (do_brk(oldbrk, newbrk-oldbrk) == oldbrk)
    mm->brk = addr;
goto out;
```

The do_brk( ) function is actually a simplified version of do_mmap( ) that handles only anonymous memory regions. Its invocation might be considered equivalent to:

```
do_mmap(NULL, oldbrk, newbrk-oldbrk, PROT_READ|PROT_WRITE|PROT_EXEC,
        MAP_FIXED|MAP_PRIVATE, 0)
```

do_brk( ) is slightly faster than do_mmap( ), because it avoids several checks on the memory region object fields by assuming that the memory region doesn't map a file on disk.